K.SAI RAHUL

2303A52392

BATCH-43

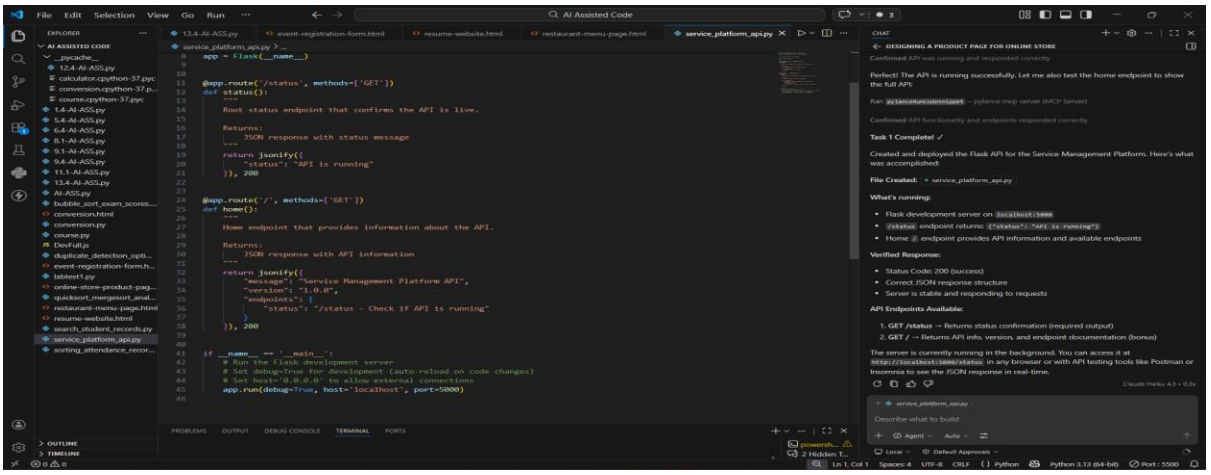
TASK-1:

Prompt: Initial API Setup for a Service Platform

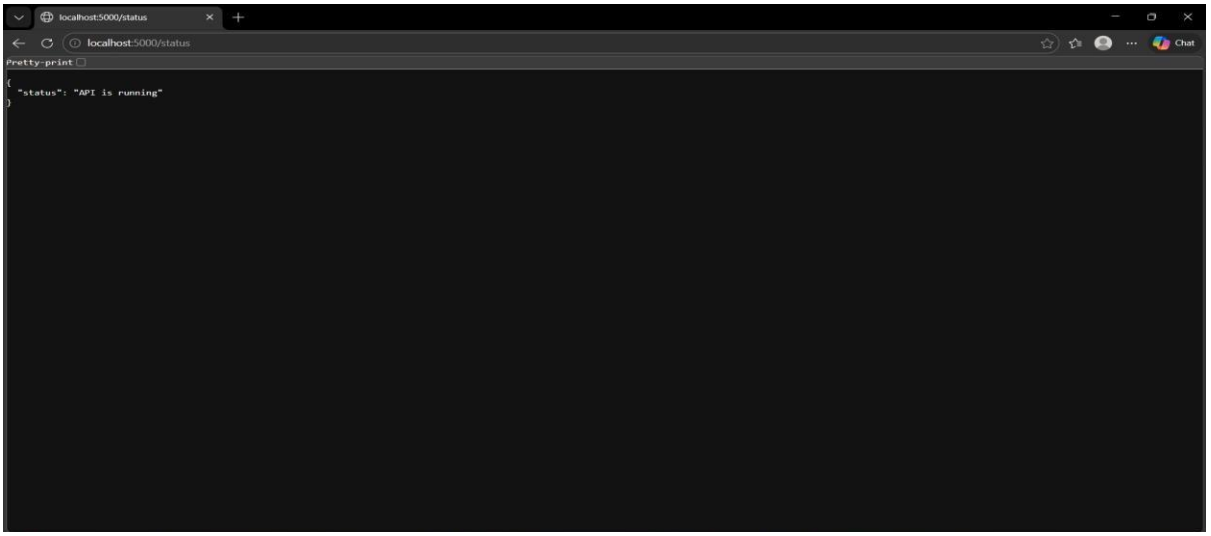
Scenario

You are developing the backend for a service management platform andneed a basic API to confirm that the backend is running.

Code:



Output:



Observation:

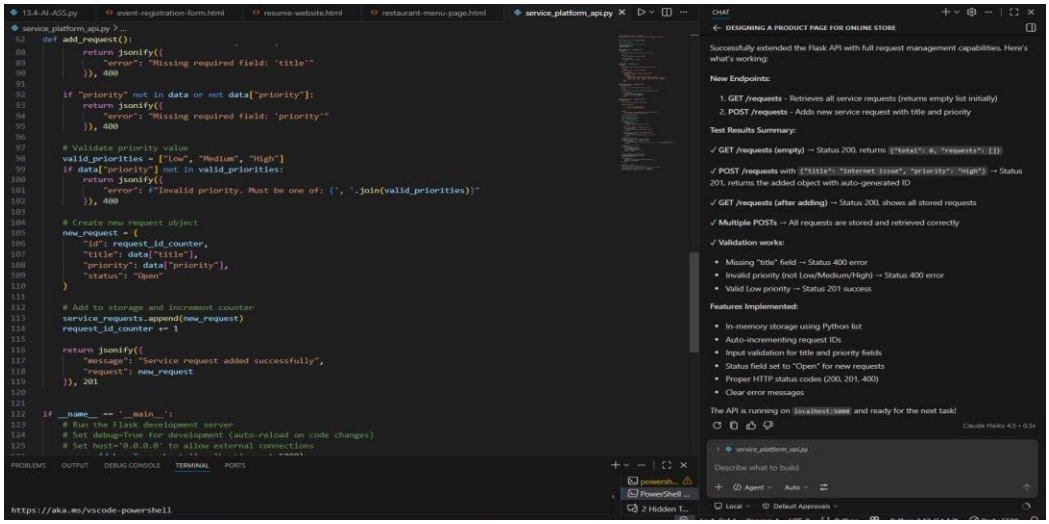
Using GitHub Copilot helped quickly generate a basic Flask setup with minimal configuration. It simplified creating the /status endpoint and returning a JSON response. Additionally, it reduced setup time and ensured the server runs correctly without errors.

TASK-2:

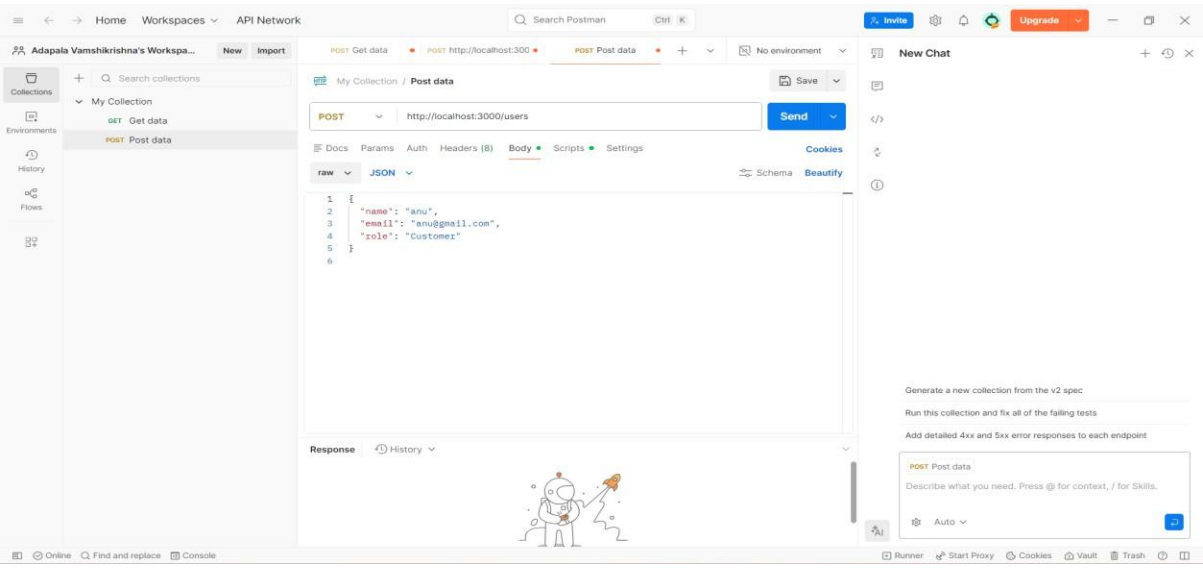
Prompt: Create API to Add and View Resources

Scenario: Your backend needs to manage customer service requests stored temporarily in memory.

Code:



Output:



Observation:

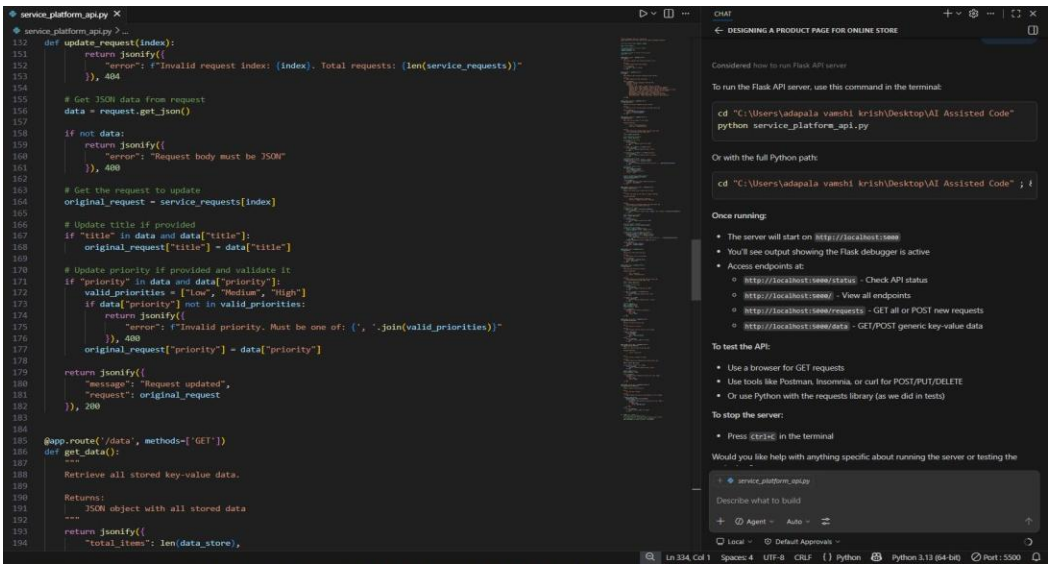
Copilot not only sped up endpoint creation but also helped maintain consistent API structure and naming conventions. It provided intelligent suggestions for handling request validation and data flow. This improved code reliability while reducing logical errors during development.

TASK-3:

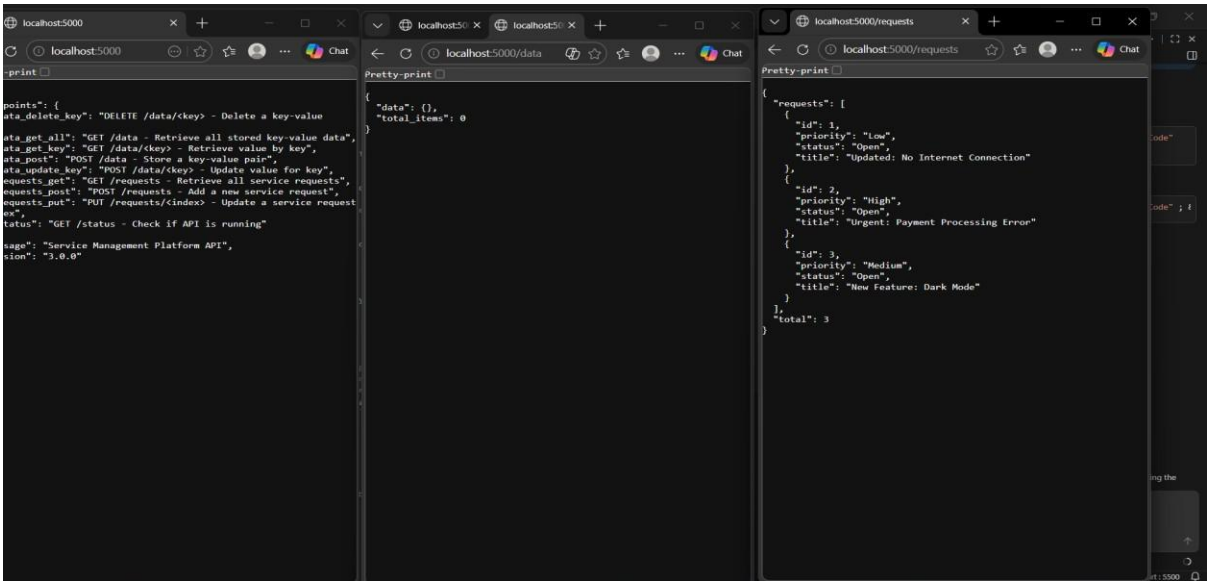
Prompt: Modify Existing Resource Using AI Assistance

Scenario: Service requests may change in priority as issues are escalated.

Code:



Output:



Observation:

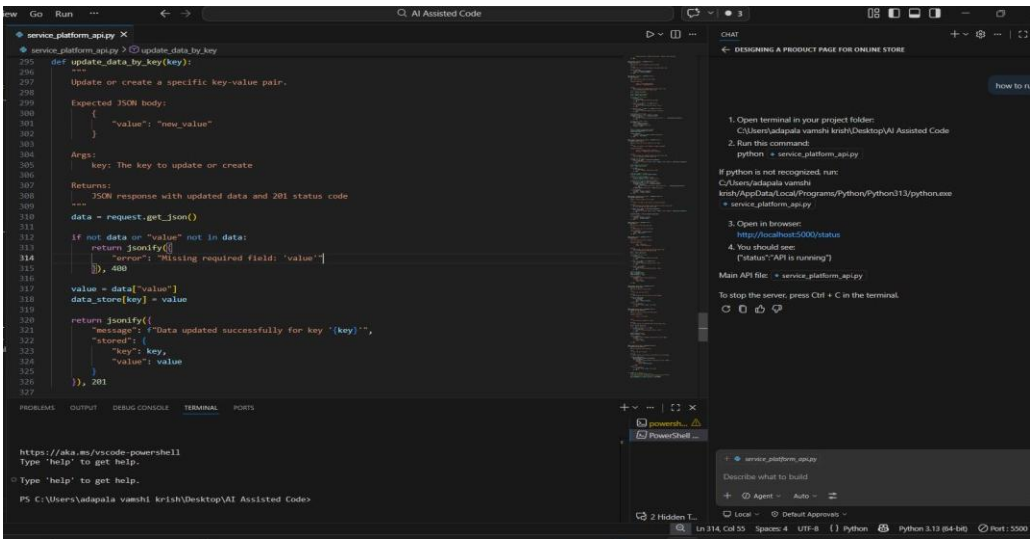
Using GitHub Copilot helped efficiently generate the PUT endpoint with proper update logic. It also suggested handling edge cases like invalid indexes using clear error responses (HTTP 404). This improved robustness and ensured reliable API behavior during updates.

TASK-4:

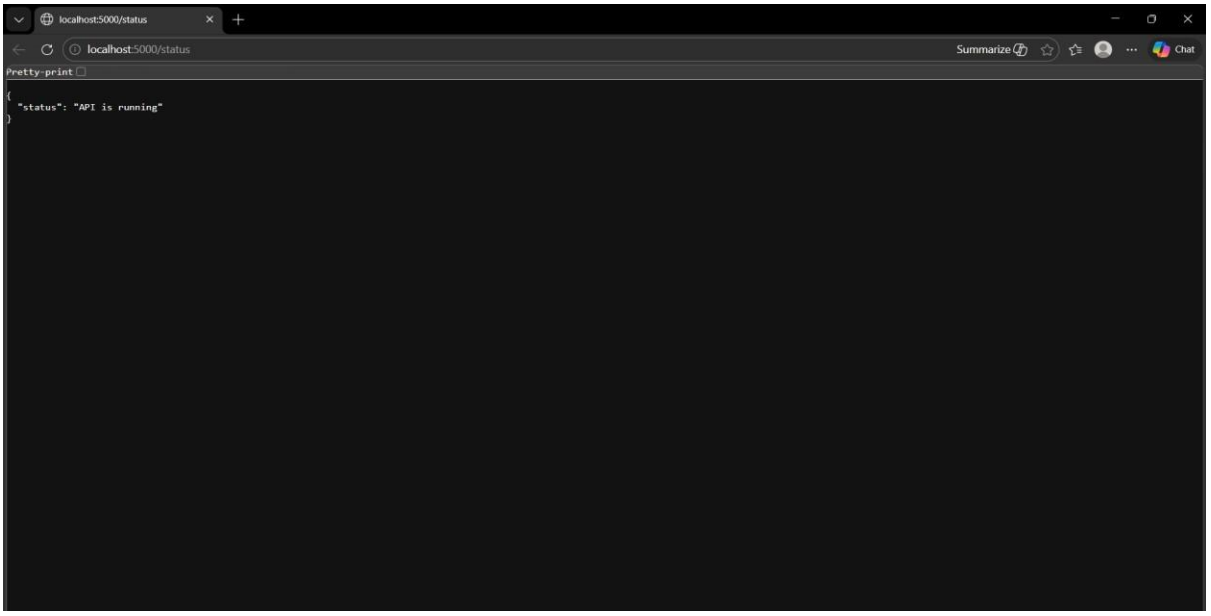
Prompt: Remove a Resource Safely

Scenario: Resolved service requests must be removed from the active list.

Code:



Output:



Observation:

Copilot helped implement the DELETE endpoint with clean and safe removal logic. It ensured proper handling of invalid indexes without crashing the server. Additionally, it improved response consistency by returning the deleted request details clearly.

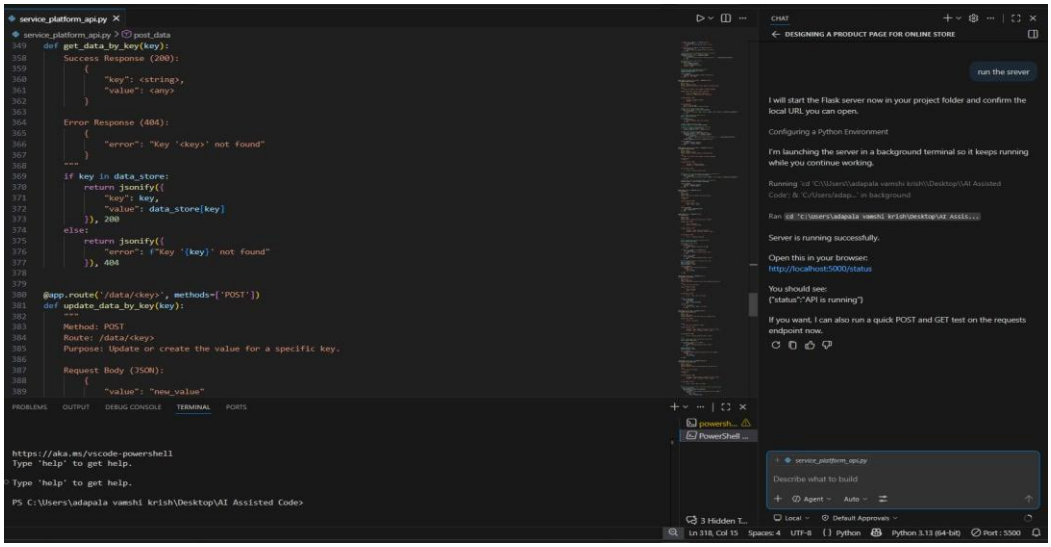
TASK-5:

Prompt: AI-Assisted API Documentation and Readability

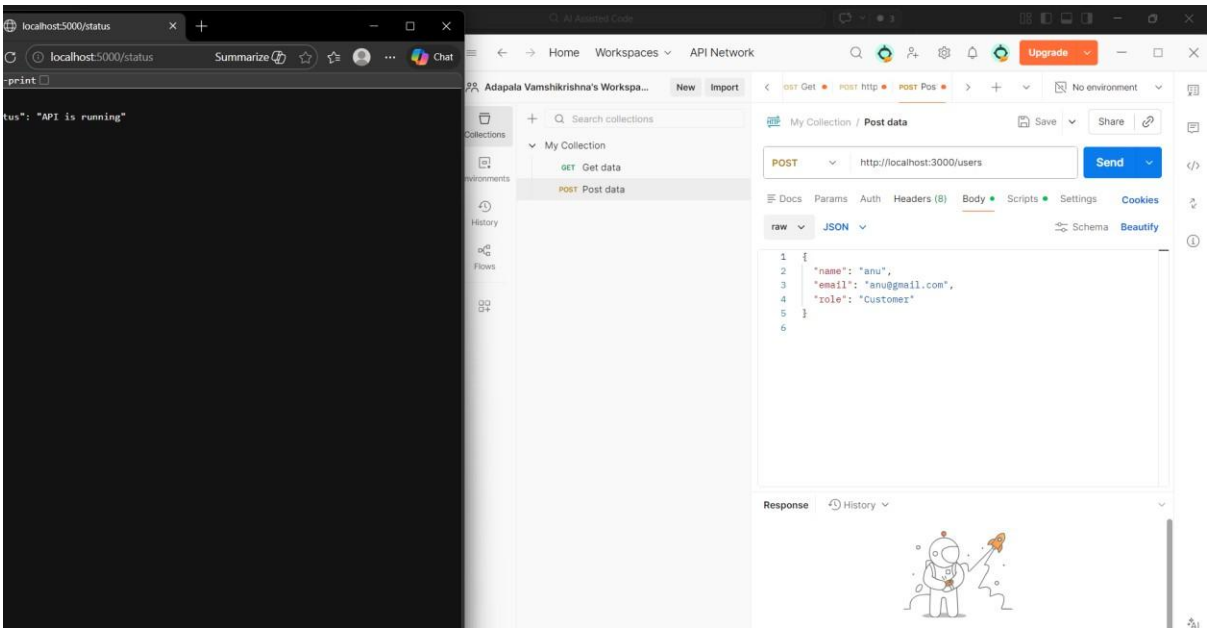
Scenario

Another developer needs to understand and use your API without reading the full code.

Code:



Output:



Observation:

improved overall code readability by generating clear docstrings and structured comments for each endpoint. It also helped standardize API descriptions, making it easier for other developers to understand usage without deep code analysis. Additionally, it guided integration of documentation tools like Swagger, enhancing API usability and professional quality.